

Multi-Tenant Learning Platform

Objective: Build a working multi-tenant learning platform for institutes/organizations. Each organization is a Tenant that can have its own administrators, users, and courses. The main goal is to demonstrate your understanding of multi-tenancy, role-based access control, data isolation, and tenant lifecycle management.

You are free to choose the technology stack and architecture, but you should be able to explain your decisions.

1. Roles & Permissions

The system should support these roles:

- **Super Admin** — platform-level administrator who can create and manage tenants.
- **Admin** — platform-level administrative user with a defined subset of Super Admin permissions.
- **Super Viewer** — platform-level read-only user.
- **Tenant Admin** — manages users and courses within their own tenant.
- **Tenant User** — learner who can access courses assigned to them.

Define the exact permissions for each role and **enforce them on the backend**, not only through the UI.

2. Learning Platform

A Tenant represents an institute.

A Tenant Admin should be able to:

- Create/manage courses.
- Add/manage users within their tenant.
- Assign courses to Tenant Users.
- View course assignments and learning progress.

A Tenant User should be able to:

- View courses assigned to them.
- Read course/lesson content.
- Track their progress/completion.

The system should support multiple tenants at the same time.

Important: Tenant Isolation

A user from Tenant A must never be able to access Tenant B's data, even by manually changing IDs in API requests. For example:

- Tenant A User → Tenant A Course ✓
- Tenant A User → Tenant B Course ✗
- Tenant A Admin → Tenant B User ✗

How you implement tenant isolation is your decision. Explain your approach in the documentation.

3. Tenant Trial & Expiration

New tenants should have a free trial. Implement tenant lifecycle handling, including:

Created → Trial Active → Expired

When a tenant expires, access to the tenant's learning functionality should be restricted appropriately.

Document:

- How the trial period is calculated.
- How expiration is detected/processed.
- What happens to existing users and data.
- How a tenant could be reactivated.
- How you handle repeated expiration processing.

You do not need to implement payments or subscriptions.

4. AI-Powered Course Creation — Design Only

A Tenant Admin should eventually be able to create a course using AI.

You do NOT need to implement the AI functionality. Instead, provide a detailed technical design explaining how you would build it.

For example, a Tenant Admin provides:

- Learner: Backend Developer
- Current Level: Beginner
- Goal: Learn Python
- Available Time: 5 hours/week
- Duration: 6 weeks

Explain how your system would use this information to generate a personalized course. Your design should cover:

- What information would be sent to the AI.
- How you would structure and validate the AI response.
- How you would personalize courses for different users.
- Problems you expect, such as hallucinations, inconsistent output, token/cost limits, privacy, prompt injection, etc.
- How you would address those problems.

Important AI Restriction

Do not use AI to write this design/documentation. The purpose of this section is to understand your own system-design thinking. Using AI-generated documentation containing terminology or architecture concepts that you cannot explain yourself may result in disqualification.

5. Testing

Include tests for the important parts of the system, especially:

- Authentication/authorization.
- Role permissions.
- Tenant isolation.
- Tenant expiration.
- Course assignment.
- Learning progress.

You do not need 100% test coverage. Focus on meaningful cases and important edge cases.

6. AI Coding Tools

You may use AI assistants while coding. If you do, please include the relevant configuration/context in the repository, such as:

- AGENTS.md
- CLAUDE.md
- .cursor/
- skills/
- .github/
- Prompts that you provide.

...or whatever mechanism you use.

You may also briefly document how you use AI, manage context, and reduce unnecessary token usage. This is optional, but good use of AI tooling and engineering workflow will be considered a plus point.

7. Submission

Submit a GitHub repository containing:

- Working application.
- Source code.
- Tests.
- Database migrations/schema.
- README with setup and run instructions.

- Short architecture/design documentation.
- AI course-generation design.
- Demo/test credentials if required.
- Estimated time and actual time taken.

8. Time & Self-Deadline

Before starting, provide your own estimated completion time and deadline.

Example:

- Estimated time: 18 hours
- Self-deadline: Friday, 6 PM

Please be realistic and responsible about the deadline you choose.

The more time you take, the more we will expect from the implementation. We would rather see a focused, well-designed, working system delivered on time than a much larger incomplete system.

If you finish earlier, submit it earlier.

What We Care About Most

Multi-tenancy → Authorization → Data isolation → Backend quality → Tenant lifecycle → Testing → System design

The UI does not need to be highly polished. Focus on building a system that works and being able to explain why you built it the way you did.